

Python Programming Notes

Units 1-3

Prepared by Course Instructor

October 10, 2025

Unit 1: Procedural Programming and Data Types (CO I)

Q1: What is Procedural Programming?

A: A paradigm using procedures or functions to execute code step-by-step. It organizes code into reusable blocks for better modularity. **Ex:** `def factorial(n): return 1 if n==0 else n*factorial(n-1); print(factorial(5))` outputs 120.

Q2: What are Identifiers and Keywords?

A: Identifiers are user-defined names for variables or functions. Keywords are reserved words like `if`, `def` with special meanings. **Ex:** `my_var = 10; import keyword; print(keyword.kwlist)` lists keywords.

Q3: Define Integral Types?

A: Integers (`int`) represent whole numbers with no limit. Booleans (`bool`) are a subtype with `True` (1) or `False` (0). **Ex:** `x = 5; print(int(True) + x)` outputs 6.

Q4: What are Floating Point Types?

A: Floats (`float`) represent real numbers with decimals. They use scientific notation and require `math.isclose()` for precise comparison. **Ex:** `y = 3.14; print(math.isclose(0.1 + 0.2, 0.3))` outputs `True`.

Q5: What is a String?

A: An immutable sequence of characters enclosed in quotes. It supports indexing and various manipulation methods. **Ex:** `s = "Hello"; print(s[0])` outputs `H`.

Q6: How to Compare Strings?

A: Strings are compared lexicographically using `<`, `==`. Comparison is case-sensitive based on Unicode values. **Ex:** `print("apple" < "banana")` outputs `True`.

Q7: What is String Slicing and Striding?

A: Slicing extracts substrings using [start:end]. Striding adds a step with [start:end:step] for custom intervals. Ex: `s = "Hello"; print(s[0:3]); print(s[::2])` outputs Hel Hlo.

Q8: What are String Operators and Methods?

A: Operators include + for concat and * for repeat. Methods like `strip()` remove spaces and `replace()` modify text. Ex: `s = " hi "; print(s.strip() + "there")` outputs hithere.

Q9: How to Format Strings with `str.format`?

A: Uses {} placeholders for dynamic insertion. Supports positional and keyword arguments for flexibility. Ex: `print("Name: 0, Age: 1".format("Alice", 30))` outputs Name: Alice, Age: 30.

Q10: What are Collection Data Types?

A: Includes tuples, lists, sets, and dictionaries for data storage. Each type offers unique features like immutability or uniqueness. Ex: `t = (1,); lst = [1]; s = 1; d = {"a": 1}`.

Q11: How to Iterate and Copy Collections?

A: Iterate with for loops over elements. Copy using `copy()` for shallow or `deepcopy()` for nested structures. Ex: `for x in [1, 2]: print(x); import copy; lst2 = copy.copy([1, 2])`.

Q12: What is PIP?

A: A package manager to install and manage Python libraries. It supports virtual environments for project isolation. Ex: `pip install requests; pip list` shows installed packages.

Unit 2: Control Structures, Functions, and Library Modules (CO II)

Q13: What are Python Control Structures?

A: Structures like sequence, selection, and iteration control flow. They include if-else, loops, and function calls for logic. Ex: `if x > 0: print("Positive")` controls output.

Q14: What are Functions and OOP Control Structures?

A: Functions encapsulate control logic for reuse. OOP integrates control within class methods and objects. Ex: `def func(): if True: print("Hi")` uses control.

Q15: What is Conditional Branching?

A: Uses `if`, `elif`, `else` for decision-making. Allows nested conditions for complex logic. Ex: `if x > 0: print("Positive") else: print("Negative")` outputs based on `x`.

Q16: What is Looping?

A: Repetition with `for` over sequences or `while` with conditions. Includes `break`, `continue`, and `else` clauses. Ex: `for i in range(3): print(i)` outputs 0 1 2.

Q17: What is Exception Handling?

A: Manages errors using `try-except` blocks. Includes `else` for success and `finally` for cleanup. Ex: `try: 1/0 except: print("Error")` outputs Error.

Q18: What are Custom Functions?

A: User-defined with `def` for specific tasks. Supports parameters, return values, and default arguments. Ex: `def add(a, b=0): return a + b; print(add(2, 3))` outputs 5.

Q19: What is the random Module?

A: Generates random numbers and selections. Includes functions like `randint()`, `choice()`, and `shuffle()`. Ex: `import random; print(random.randint(1, 10))` outputs a random int.

Q20: What is the math Module?

A: Provides mathematical functions like `sqrt()` and `factorial()`. Includes constants like `pi` and advanced operations. Ex: `import math; print(math.sqrt(16))` outputs 4.0.

Q21: What is the time Module?

A: Handles time-related tasks like timestamps. Offers `sleep()` and formatted time outputs. Ex: `import time; print(time.time())` outputs current timestamp.

Q22: What is the os Module?

A: Interacts with the operating system environment. Supports directory and file operations. Ex: `import os; print(os.getcwd())` outputs current directory.

Q23: What is the shutil Module?

A: Performs high-level file and directory operations. Includes copying, moving, and removing files. Ex: `import shutil; shutil.copy("a.txt", "b.txt")` copies file.

Q24: What is the sys Module?

A: Accesses system-specific parameters and functions. Manages command-line arguments and exit codes. Ex: `import sys; print(sys.version)` outputs Python version.

Q25: What is the glob Module?

A: Matches file patterns using wildcards. Supports recursive directory searches. Ex: `import glob; print(glob.glob("*.txt"))` lists .txt files.

Q26: What is the re Module?

A: Handles regular expressions for pattern matching. Supports searching, replacing, and splitting text. Ex: `import re; print(re.search(r"1", "a1b2").group())` outputs 1.

Q27: What is the statistics Module?

A: Computes statistical measures like mean and median. Includes variance and mode for data analysis. Ex: `import statistics; print(statistics.mean([1, 2, 3]))` outputs 2.0.

Unit 3: OOP and File Handling (CO III)

Q28: What is Creating a Custom Module?

A: A .py file containing reusable code and functions. Can be imported and executed with `if __name__ == "__main__":`. Ex: `mymod.py: def greet(): print("Hi"); import mymod; mymod.greet()` outputs Hi.

Q29: What are Object Oriented Concepts?

A: Includes encapsulation, inheritance, and polymorphism. Abstraction hides complexity for better design. Ex: `class A: pass; a = A()` creates object.

Q30: What are Custom Classes?

A: User-defined blueprints using class. Instances are created to represent objects. Ex: `class Person: pass; p = Person()` creates instance.

Q31: What are Attributes and Methods?

A: Attributes store data within objects. Methods are functions that operate on object data. **Ex:** `class P: def init(self):self.x=1;p=P();print(p.x)` outputs 1.

Q32: What is Inheritance and Polymorphism?

A: Inheritance reuses parent class code. Polymorphism allows method overriding in subclasses. **Ex:** `class C(P): def m(self): print("C"); c = C(); c.m()` outputs C.

Q33: What is Using Properties?

A: Controls attribute access with @property decorators. Includes setters and deleters for data management. **Ex:** `class C: @property def x(self): return 1; print(C().x)` outputs 1.

Q34: What is Writing and Reading Binary Data?

A: Handles binary files using 'b' mode for non-text data. Ensures proper reading and writing of bytes. **Ex:** `with open("f.bin", "wb") as f: f.write(b"data"); print(open("f.bin", "rb").read())` outputs b'data'.

Q35: What is Writing and Parsing Text Files?

A: Manages text data with read and write operations. Parsing involves splitting or processing text lines. **Ex:** `with open("f.txt", "w") as f: f.write("hi"); print(open("f.txt").read())` outputs hi.